

UT 01.1 · DISEÑO WEB EN ENTORNO CLIENTE

TypeScript I

Introducción, tipos básicos y el compilador de TypeScript

¿Qué vamos a ver en esta unidad?

01

Introducción a TypeScript

Qué es, sus ventajas frente a JavaScript y cómo configurar el primer proyecto.

02

Tipos en TypeScript

number, string, boolean, objetos, arrays, funciones, uniones, literales, enums e interfaces.

03

El compilador (tsc)

Modo watch, tsconfig.json, y cómo organizar el proyecto con rootDir y outDir.

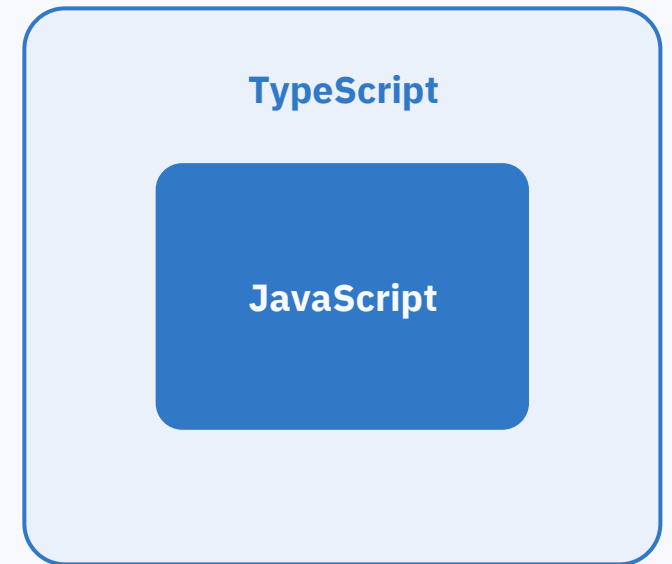
01

Introducción a TypeScript

Qué es, sus ventajas y la configuración del primer proyecto

¿Qué es TypeScript?

- Es un **superconjunto sintáctico** estricto de JavaScript.
- Añade **tipado estático** a un lenguaje que, por defecto, es de tipado dinámico.
- Todo el código JavaScript existente es **válido en TypeScript**.
- El código TypeScript, en cambio, **no es válido** en JavaScript: aporta características adicionales.



Ventajas de TypeScript

Detecta errores antes de ejecutar

- El compilador detecta errores de tipos **al instante**, mientras escribes.
- En JavaScript (tipado dinámico), un error como llamar a `array.map` sobre un número solo se detecta en **tiempo de ejecución**, cuando el programa ya ha fallado.
- Puedes usar las últimas características de JavaScript evitando erratas antes de que se produzcan.

LIMITACIÓN

Los navegadores **solo entienden JavaScript**.

TypeScript debe **compilarse a JavaScript** antes de poder ejecutarse.

01 · INTRODUCCIÓN

Configuración del primer proyecto

Crea en VS Code un [index.html](#) básico que haga referencia a un archivo [main.js](#).

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Conceptos básicos de TypeScript</title>
  </head>
  <body>
    <h1>Conceptos básicos de TypeScript</h1>
    <script src="main.js"></script>
  </body>
</html>
```

Instala TypeScript de forma global (añade **sudo** en Mac/Linux):

```
$ npm i -g typescript
```

Trabajar con npm y el comando tsc

npm

- Es el gestor de paquetes **predeterminado** de Node.js.
- En **npmjs.com** no hay proceso de selección ni control de calidad: cualquiera puede publicar.
- Comprobar la versión instalada:

```
$ npm -v
```

main.ts

```
console.log('Hola TypeScript');
```

```
$ tsc main.ts
```

```
// main.js generado en el mismo directorio
```

Live Server y resumen del capítulo

Extensión Live Server

Recarga el navegador automáticamente en cada cambio, para no tener que volver a ejecutar el proyecto manualmente.

RESUMEN

- Instalaste TypeScript de forma global.
- Creaste archivos básicos de HTML y TypeScript.
- Conseguiste que main.ts y main.js funcionen juntos.

02

Conceptos básicos de TypeScript

Los tipos, sus reglas, y cómo usarlos en tus proyectos

El tipo «number»

- `myNum` no lleva anotación: TypeScript **infiere** el tipo `number` a partir de su valor.
- `anotherNum` declara explícitamente el tipo **`number`**.
- Ninguna de las dos variables puede contener algo que no sea un número.
- El error se marca **antes de ejecutar** el código, al escribirlo.

index.ts — Listado 2-1

```
// Numbers
let myNum = 10;
let anotherNum: number = 20;

myNum = 12;
myNum = '12'; // Error
anotherNum = 30;
anotherNum = false; // Error
```

Los tipos «string» y «boolean»

- Funcionan igual que **number**: si asignas un tipo y luego uno incorrecto, aparece un error.
- Si no se especifica un tipo, TypeScript lo **infiere** a partir del valor: `anotherStr` → `string`, `anotherBool` → `boolean`.
- Esa inferencia también se aplica al reasignar valores incompatibles.

index.ts — Listado 2-2

```
// String
let myStr: string = 'Hello';
let anotherStr = 'World';
myStr = true; // Error
anotherStr = 45; // Error

// Boolean
let myBool: boolean = true;
let anotherBool = false;
myBool = 'true'; // Error
```

Inferencia: ¿cuándo asignar un tipo?

En la mayoría de los casos, deja que TypeScript infiera el tipo. Pero si vas a asignar un valor más adelante, especifícalo explícitamente.

Sin tipo explícito — Listado 2-3

```
let salary;  
salary = 12000;  
salary = '12000';  
salary = true;
```

Sin error: acepta números, cadenas y booleanos.

Con tipo explícito — Listado 2-4

```
let salary: number;  
salary = 12000;  
salary = '12000'; // Error  
salary = true; // Error
```

Con tipo: TypeScript detecta las asignaciones incorrectas.

02 · TIPOS

Objetos

Al especificar el tipo de cada clave, TypeScript avisa si asignas un valor distinto o una clave que no existe.

Sin tipos — Listado 2-5

```
const developer = {  
  firstName: 'Nabendu',  
  lastName: 'Biswas',  
  age: 40,  
  isTrainer: true  
}
```

Con tipos — Listado 2-6

```
const newDeveloper: { name: string; age: number; isDev: boolean } = {  
  name: 'Mousam', age: 39, isDev: true  
}  
newDeveloper.name = 'Mousam Mishra';  
newDeveloper.age = 'Forty'; // Error  
newDeveloper.firstName = 'Mousam'; // Error: no existe
```

Matrices (arrays)

- Si se proporciona un array de cadenas, no se puede añadir un número ni un booleano.
- También puedes declarar explícitamente el tipo: `number[]`.
- En ambos casos, TypeScript restringe qué se puede añadir con `.push()`.

index.ts — Listado 2-7

```
const languages = ['React', 'Angular', 'Vue'];
languages.push('TypeScript');
languages.push(56); // Error
languages.push(true); // Error

const numbers: number[] = [51, 22, 33];
numbers.push(56);
numbers.push('56'); // Error
```

Matrices complejas

Un array puede contener objetos tipados, o incluso otros arrays: se usa doble corchete `[][]`.

Array de objetos — Listado 2-8

```
const arrOfObj: { name: string; age: number }[] = [
  { name: 'Nabendu', age: 40 },
  { name: 'Mousam', age: 39 }
];
arrOfObj.push({ name: 'Shikha', age: 39 });
arrOfObj.push({ name: 'Hriday', age: 'Eight' });
```

Array de arrays — Listado 2-9

```
const arrOfArrays: number[][] = [
  [11, 32, 43],
  [34, 75, 64]
];
arrOfArrays.push([21, 32, 13]);
```

Funciones: tipar los parámetros

Si no especificas el tipo de los parámetros, no se produce ningún error aunque pases una cadena. Siempre debes tiparlos.

Sin tipos — Listado 2-10

```
// Funciones
const addNums = (num1, num2) => {
  return num1 + num2;
}
addNums(10, 20);
addNums(10, '20');
```

Sin error, aunque '20' sea una cadena: bug silencioso.

Con tipos — Listado 2-11

```
const multiNums = (num1: number, num2: number) => {
  return num1 * num2;
}
multiNums(10, 20);
multiNums(10, '20'); // Error
```

Con tipos, TypeScript detecta el argumento incorrecto.

Funciones: el tipo de retorno

También es recomendable especificar el tipo de retorno: así se detecta el error si el valor devuelto no coincide. Si no devuelve nada, se usa `void`.

Retorno tipado — Listado 2-11

```
const modNums = (num1: number, num2: number): number => {  
  return num1 > num2; // Error: es boolean  
}  
modNums(10, 20);
```

El tipo de retorno atrapa también errores dentro de la función.

Sin retorno: void — Listado 2-11

```
const printSum = (num1: number, num2: number): void => {  
  console.log(num1 + num2);  
}  
printSum(10, 20);  
printSum(10, '20'); // Error
```

`void` indica que la función no devuelve ningún valor.

Tipos de unión

- Con `|` una variable puede tener **varios tipos posibles**.
- `numOrStr` puede ser `number` o `string`.
- También puedes tener un array que solo admita elementos de esos tipos.

index.ts — Listado 2-12

```
// Tipos de unión
let numOrStr: number | string;
numOrStr = 10;
numOrStr = 'Ten';

let arr: (number | string)[] = [10, 'Ten', true]; // Error
```

Tipos literales

- Solo se especifican los **valores exactos** que se aceptan.
- En el ejemplo, únicamente 'Nabendu', 'Mousam', 'Shikha' o 'Hriday' son válidos.
- Cualquier otro valor, como 'Parag', produce un **error**.

index.ts — Listado 2-13

```
let myLiteral:  
  'Nabendu' | 'Mousam' | 'Shikha' | 'Hriday'  
  = 'Nabendu';  
myLiteral = 'Mousam';  
myLiteral = 'Shikha';  
myLiteral = 'Hriday';  
myLiteral = 'Parag'; // Error
```

Tipos enumerados (enum)

- **enum** combina la idea de tipo de unión y tipo literal.
- Define un conjunto fijo de valores con nombre: ADMIN, READ_ONLY, AUTHOR.
- Una vez definido, puedes usarlo en variables y anotaciones de tipo.

index.ts — Listado 2-14

```
// Enumeración
enum Role { ADMIN, READ_ONLY, AUTHOR };

const myRole = Role.ADMIN;
const hridayRole: Role = Role.AUTHOR;
```

Tipos opcionales

Un campo opcional, con `?`, no es obligatorio, pero si se proporciona debe cumplir su tipo.

Unión con undefined

```
let optionalObj: {  
  name: string;  
  age: number | undefined  
} = {  
  name: 'Nabendu',  
  age: undefined  
};
```

Hay que dejar «age» indefinido explícitamente.

Con el operador ?

```
let betterOptObj: {  
  name: string;  
  age?: number  
} = {  
  name: 'Nabendu'  
};
```

«age» no es obligatorio: puede omitirse por completo.

Interfaces

- Forma más adecuada de definir el tipo de las propiedades de un objeto.
- Se declara una vez y se **reutiliza** en tantas variables como necesites.
- `person1` y `person2` comparten la misma interfaz `Developer`.

index.ts — Listado 2-16

```
interface Developer {  
  name: string;  
  age: number;  
  isDev: boolean;  
}  
  
const person1: Developer = { name: 'Nabendu', age: 40, isDev: true }  
const person2: Developer = { name: 'Mousam', age: 39, isDev: true }
```

Tipos (type) vs. interfaces

Se puede usar en...	interface	type
Objetos	Sí	Sí
Strings / uniones / arrays	No	Sí

`CoderType` incluso puede ser un array de objetos con una unión literal: `category: 'frontend' | 'backend' | 'mobile'`.

index.ts — Listado 2-17

```
type DeveloperType = {  
  name: string; age: number; isDev: boolean;  
}  
const person3: DeveloperType = { name: 'Nabendu', age: 40, isDev: true }  
  
type PersonName = string;  
const person4: PersonName = 'Nabendu';
```

Los tipos de TypeScript, de un vistazo

number

Valores numéricos: 10, 3.14...

string

Cadenas de texto.

boolean

true o false.

objeto

Tipa cada clave del objeto.

T[]

Array de un tipo, o de arrays.

función

Parámetros y retorno tipados.

A | B

Tipo de unión: varios tipos posibles.

literal

Solo valores exactos permitidos.

enum

Conjunto fijo de valores con nombre.

campo?

Opcional: puede faltar.

interface

Define la forma de un objeto, reutilizable.

type

Alias de tipo: objetos, strings, uniones...

03

El compilador de TypeScript

Modo watch, tsconfig.json y organización del proyecto

Modo de vigilancia (watch)

- Supervisa los cambios en el archivo y regenera el **.js** automáticamente.
- Ya no hace falta ejecutar **tsc** después de cada cambio.
- Se activa con el indicador **-w**.

```
$ tsc main.ts -w
```

main.ts — Listado 3-1

```
type PersonNewName = string;  
const person6: PersonNewName = 'Mousam';  
console.log(`New Developer is ${person6}`);
```

Compilar un proyecto completo

Para compilar todos los archivos de TypeScript a la vez, conviertes la carpeta en un proyecto de TypeScript.

```
$ tsc --init
```



Crea `tsconfig.json` en la raíz del proyecto.

```
$ tsc
```



Compila **todos** los archivos `.ts` según ese archivo.

Problema: `index.ts` contiene muchos errores y detiene la compilación de todo el proyecto.

exclude e include

- `exclude` quita archivos o carpetas de la compilación, como **`index.ts`** (lleno de errores) o **`node_modules`**.
- `node_modules` puede contener código TypeScript de terceros que no quieres compilar.
- `include` añade explícitamente los archivos que sí quieres compilar.

tsconfig.json

```
{  
  "exclude": ["index.ts", "node_modules"],  
  "include": ["main.ts"]  
}
```

rootDir y outDir

- Guarda todos los archivos .ts en una carpeta **src**.
- Crea una carpeta **dist** para el JavaScript generado.
- Descomenta **rootDir** y **outDir** en tsconfig.json e indica esas carpetas.
- Actualiza la ruta de **main.js** en index.html.

Estructura

```
proyecto/  
├─ src/  
│   └─ index.ts  
│       └─ main.ts  
├─ dist/  
│   └─ main.js  
└─ index.html
```

```
"rootDir": "./src",  
"outDir": "./dist"
```

De JavaScript dinámico a TypeScript tipado

01

TypeScript compila a JavaScript; instalamos y configuramos el primer proyecto.

02

Tipos básicos, objetos, arrays, funciones, uniones, literales, enums e interfaces.

03

El compilador tsc: modo watch, tsconfig.json, rootDir y outDir.